

SEER: Enhancing Chain-of-Thought Code Generation through Self-Exploring Deep Reasoning

Shuzheng Gao

The Chinese University of Hong Kong
Hong Kong, China
szgao23@cse.cuhk.edu.hk

Cuiyun Gao*

Harbin Institute of Technology, Shenzhen
Shenzhen, China
gaocuiyun@hit.edu.cn

Chaozheng Wang

The Chinese University of Hong Kong
Hong Kong, China
czwang23@cse.cuhk.edu.hk

Michael R. Lyu

The Chinese University of Hong Kong
Hong Kong, China
lyu@cse.cuhk.edu.hk

Abstract

Code generation has recently seen tremendous advances through Chain-of-Thought (CoT) reasoning, which enables Large Language Models (LLMs) to develop high-level reasoning plans before writing code. Recent research has proposed various methods to enhance models' CoT reasoning for code generation such as prompt engineering and supervised fine-tuning. However, existing approaches still face three critical limitations: (1) limited exploration of diverse reasoning paths, which constrains generalization across various programming scenarios, (2) lack of quality assessment for intermediate reasoning steps, which hampers the reliability of the generated plans and code, and (3) the potential negative impact of "overthinking", potentially leading to unnecessarily complex and incorrect solutions. To address these limitations, we frame CoT code generation as a decision making problem and present SEER, a **Self-Exploring deep Reasoning** framework that enables accurate and adaptive reasoning for code generation. SEER introduces three key components: (1) Diverse reasoning path exploration, which aims at exploring diverse reasoning paths and annotating intermediate steps without relying on manual experts or closed-source proprietary models; (2) Reasoning quality-aware model training, which trains a policy model for generating candidate reasoning steps and a value model for assessing their quality; and (3) Adaptive CoT reasoning, which dynamically switches between direct generation and step-by-step reasoning for different problems. Experiments on state-of-the-art code LLMs demonstrate that SEER achieves remarkable performance gains across three popular code generation benchmarks, consistently outperforming all baseline methods and achieving absolute improvements by 4.2% ~ 9.3% in MBPP, 1.9% ~ 9.1% in HumanEval and 3.5% ~ 5.3% in LiveCodeBench, respectively.

ACM Reference Format:

Shuzheng Gao, Chaozheng Wang, Cuiyun Gao, and Michael R. Lyu. 2026. SEER: Enhancing Chain-of-Thought Code Generation through Self-Exploring Deep Reasoning. In *2026 IEEE/ACM 48th International Conference*

*Corresponding author.



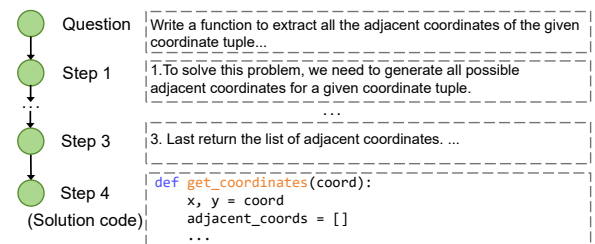
This work is licensed under a Creative Commons Attribution 4.0 International License. ICSE '26, Rio de Janeiro, Brazil

© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2025-3/26/04
<https://doi.org/10.1145/3744916.3773197>

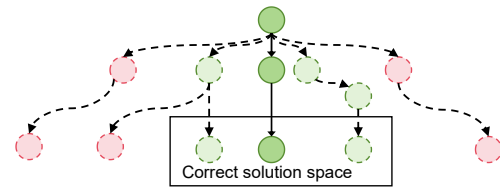
on Software Engineering (ICSE '26), April 12–18, 2026, Rio de Janeiro, Brazil. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3744916.3773197>

1 Introduction

Code generation aims at automatically generating source code that adheres to a given programming problem [28, 34]. As software systems become increasingly complex and the demand for skilled developers continues to increase, automated code generation holds great potential to advance the software development process by reducing repetitive manual programming efforts and improving productivity [13, 17, 41, 49]. Recent advances in Large Language Models (LLMs) have driven considerable progress in software engineering tasks [16, 18, 33, 51, 54–56], such as ChatGPT [6] and Deepseek-Coder [22]. Despite these success cases, producing accurate code for complex problems remains an important challenge that requires sophisticated reasoning to align implementation with problem specifications [30, 64].



(a) Chain-of-thought code generation



(b) An illustration of multiple reasoning paths

Figure 1: An illustration of chain-of-thought code generation (a) and multiple reasoning paths (b). The green and red node denote correct and incorrect steps, respectively. The dashed lines and circles represent unexplored paths.

Recently, Chain-of-Thought (CoT) [31, 52] has demonstrated great potential in improving the reasoning abilities of LLMs for

complex tasks by breaking problems into intermediate steps and solving them step by step. As shown in Figure 1 (a), CoT enhances code generation by enabling models to explicitly outline a high-level reasoning plan before producing the code, which has been shown to be effective [30]. Recent research has explored various CoT methods for code generation, with some leveraging prompt engineering and in-context learning to elicit reasoning chains during inference time [15, 19, 30, 33]. However, these techniques rely on large proprietary models and struggle with smaller models, as the latter typically lack the capacity to learn reasoning processes from prompts alone. To address this limitation, recent studies have explored distilling high-quality reasoning chains from advanced proprietary models such as GPT-4 [41] and using these curated reasoning data to train lightweight language models capable of performing CoT reasoning [60].

Despite these advances, current CoT code generation approaches still face the following critical limitations: (1) First, they **lack effective exploration of diverse reasoning paths**. Although programming problems often permit multiple valid solutions, as shown in Figure 1 (b), existing approaches typically rely on training data that contain only a single reasoning path (indicated by the solid line) due to the high cost of distilling proprietary models. This leads to models overfitting to specific reasoning patterns, constraining their ability to generalize across various problem-solving scenarios. (2) Second, these approaches **lack quality assessment of intermediate reasoning steps**. Generating reasoning based on flawed intermediate steps often leads to incorrect final solutions. Existing approaches solely train models with correct reasoning paths, ignoring the importance of step-level correctness. The oversight prevents models from learning to evaluate reasoning quality and prioritize promising paths. However, collecting step-level correctness data presents significant challenges. While final solution quality can be validated through test cases, automatically evaluating intermediate reasoning steps remains difficult, as even incorrect solutions may contain valid intermediate reasoning components. (3) Third, previous work has **overlooked the negative impact of “overthinking”**. Although CoT reasoning generally improves overall performance, we find that it can also lead models to pursue unnecessarily complex and incorrect solutions that they are not familiar with, even for relatively simple problems. For example, as shown in Figure 2 (a), although CoT reasoning improved DeepSeek-Coder-6.7B-Instruct’s performance on MBPP [2] by 7.5%, it also caused 4.7% of previously correct answers to become incorrect (details referring to Section 5.2). This not only results in performance degradation on problems that could have been straightforwardly solved (e.g. Figure 2 (b) and (c)) but also increases computational costs through higher token consumption.

To address these limitations, we frame CoT code generation as a decision-making problem and present SEER, a **SELF**-Exploring deep Reasoning framework that enables accurate and adaptive reasoning for code generation. SEER contains three stages: 1) Diverse reasoning path exploration, which aims at exploring diverse reasoning paths and annotating intermediate steps by customizing Monte Carlo Tree Search with reasoning path perturbation and refinement, eliminating the need for manual experts or proficiency models; 2) Reasoning quality-aware model training, which trains a policy model for generating candidate reasoning steps and a

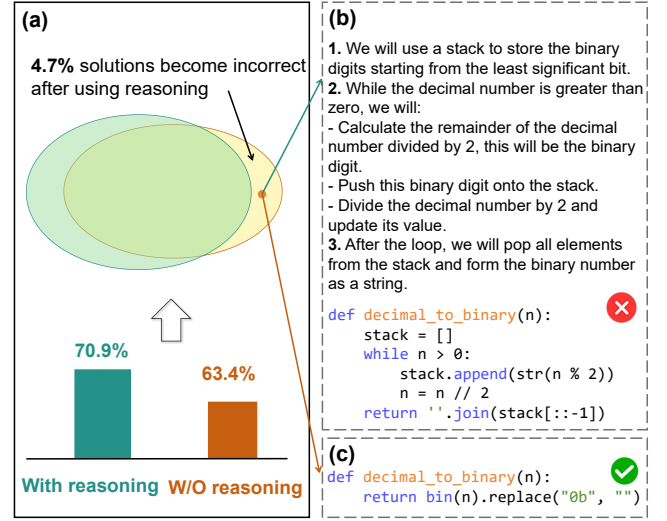


Figure 2: An example of “overthinking” in code generation: Reasoning improves overall accuracy but can also lead to errors (a). While solution (b) includes detailed reasoning, it opts for a complex implementation and fails to handle zero input. In contrast, solution (c) takes a direct approach using the built-in `bin()` function and correctly handles all inputs.

value model for evaluating their quality using self-explored data; 3) Adaptive CoT reasoning, which dynamically selects between direct generation and step-by-step reasoning for different problems.

To evaluate the effectiveness of SEER, we conduct extensive experiments on two state-of-the-art code LLMs, including DeepSeek-Coder [22] and Qwen2.5-Coder [25]. Experimental results demonstrate that SEER outperforms all baseline methods such as DeepSeek-R1-Zero’s reinforcement learning [21] and improves the base model by 4.2% ~ 9.3%, 1.9% ~ 9.1%, and 3.5% ~ 5.3% in terms of pass@1 on three popular coding benchmarks: MBPP [2], HumanEval [8], and LiveCodeBench [27], respectively. Furthermore, when enhanced with SEER, DeepSeek-Coder-6.7B outperforms models with 4-10 times more parameters, such as Llama3-70B [13] and DeepSeek-Coder-33B [22]. Furthermore, Qwen2.5-Coder-7B+SEER achieves competitive results compared to some closed-source proprietary models, including GPT-4-Turbo [41] and Gemini 1.5 Flash 002 [1].

In general, we make the following contributions:

- (1) To the best of our knowledge, we are the first to frame CoT code generation as a decision-making problem and enable autonomous exploration of step-annotated reasoning paths without relying on manual expert annotations or proprietary models.
- (2) We propose SEER, a framework that enhances code generation through diverse reasoning path exploration, reasoning quality-aware model training, and adaptive CoT reasoning.
- (3) Extensive experimentation demonstrate that SEER improves the performance of state-of-the-art LLMs by a large margin across multiple popular benchmarks and enables smaller LLMs to outperform larger ones.

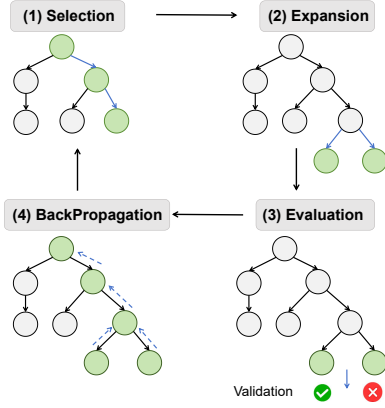


Figure 3: An overview of the four key operations in MCTS.

- (4) To facilitate further research, we publicly release the code, trained model, and step-annotated dataset on GitHub [26].

2 Preliminary

2.1 Problem Formulation

We formulate chain-of-thought code generation as a decision-making problem using the Markov Decision Process (MDP) [14]. In the MDP framework, an agent begins from an initial state s_0 and takes sequential actions a_t , transitioning from state s_{t-1} to s_t at each timestep until reaching a final state s_n that contains the solution. For code generation, states and actions are represented as token sequences, with the initial state $s_0 = x_0, \dots, x_L$ being the input prompt, where L denotes the prompt length. Following [30] and as illustrated in Figure 1, we use a single implementable subtask to solve this question as a step and the last step will give the final solution code. At each timestep t , given the prompt and current steps as state $s_t = (s_0, a_1, \dots, a_{t-1})$, a new step is generated as action $a_t = (y_{t,0}, \dots, y_{t,n_t})$, where $y_{t,i}$ is a single token and n_t is the length of the action's content. The state transition function deterministically updates the state through token concatenation: $s_{t+1} = \text{Cat}(s_t, a_t)$.

Given the large action space in the LLM's step-by-step generation process [21], our primary goal is to develop two models: a step-level policy model $\pi_\theta(s)$ and a value model $V_\phi(s)$. The policy model generates some promising actions at each state, while the value model assesses the expected returns from the current partial solution to guide the LLM in selecting the most reasonable subsequent reasoning step. Therefore, to train these models, we need to collect diverse data that contain both correct and incorrect solution paths, with annotations for intermediate reasoning steps that include both action a and value v indicating action quality.

2.2 Monte Carlo Tree Search

Monte Carlo Tree Search (MCTS) is a heuristic search algorithm that has gained noticeable attention since its breakthrough application in AlphaGo [46]. It combines tree search with Monte Carlo sampling, offering a powerful approach for sequential decision-making problems, especially those with large state spaces where traditional

methods such as the minimax search become computationally intractable [11, 47]. As shown in Figure 3, MCTS iteratively builds a search tree through four key steps: (1) **Selection**: traversing the tree from root to leaf and selecting promising nodes to expand; (2) **Expansion**: adding new child nodes to expand the tree based on available actions in the current state; (3) **Evaluation**: estimating the value of leaf nodes through random rollouts to terminal states or neural network value approximation; and (4) **Backpropagation**: updating node statistics (visit counts and action values) along the traversed path to improve future selection. Recent research has extended MCTS beyond its traditional gaming domain, exploring its potential for LLM reasoning [3, 4]. Specifically, researchers have demonstrated success in using MCTS to simulate expert-like problem solving and annotate step-by-step reasoning processes for tasks such as mathematical reasoning and proof generation [7, 59].

3 Proposed Approach

The overall procedure of SEER is illustrated in Figure 4. The framework consists of three main phases. In the **Diverse reasoning path exploration** phase, we collect high-quality and diverse reasoning data with detailed annotations of reasoning step quality, for subsequent model training. We start with a seed dataset $D_{seed} = (q^i, c^i, u^i)_{i=1}^N$ containing programming problems, code solutions, and test cases. For each sample, we first employ MCTS to construct reasoning trees through iterative selection, expansion, evaluation, and backpropagation. We then propose to enhance dataset diversity through *path perturbation* for unexplored incorrect solutions and *path refinement* for unexplored correct solutions. Based on the self-explored data, in the **Reasoning quality-aware model training** phase, we develop two models: a policy model trained on correct reasoning paths through reinforced fine-tuning to produce effective reasoning, and a value model trained on both correct and incorrect reasoning paths to assess reasoning step quality. Finally, in the **Adaptive CoT reasoning** phase, we teach the model to dynamically switch between direct generation and step-by-step reasoning for different problems. Specifically, we design an adaptive reasoning step search algorithm, where the value model is continually fine-tuned and collaborates with the policy model to choose the most suitable reasoning approach.

3.1 Diverse Reasoning Path Exploration

This part aims at collecting high-quality and diverse reasoning data with detailed annotations of reasoning step quality for model training. We first perform customized MCTS for initial data collection, followed by path perturbation and refinement to further improve the diversity of reasoning data.

3.1.1 Monte Carlo Tree Search (MCTS). As shown in Figure 3, during MCTS, SEER starts with the initial state as its root and systematically grows the search tree by adding new nodes. Specifically, within the context of code generation, we customize the four key operations of the MCTS algorithm as follows:

Selection. During each iteration of the MCTS, the selection process begins with s_0 , representing the initial state containing the input problem q . The algorithm then proceeds to explore the tree $\mathcal{T}_k$ and selects the children node with the highest PUCT score [43]. The PUCT score incorporates the log-probability from pre-trained

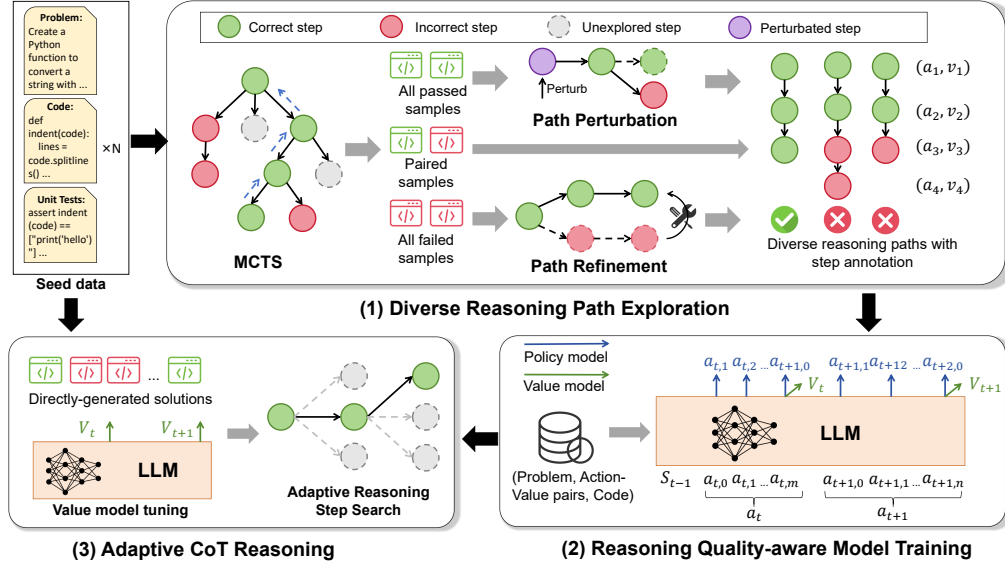


Figure 4: The overall framework of SEER.

LLM as prior knowledge to rank each node and balance exploration and exploitation during sampling based on visiting history. This selection process is mathematically represented as:

$$a_t = \arg \max_{a \in \mathcal{A}_k} [\hat{Q}(s, a) + c_{\text{puct}} \pi_{\theta_k}(a|s_t) \sqrt{\frac{N_{\text{parent}}(a)}{1 + N(s_t, a)}}] \quad (1)$$

where the state-action value $\hat{Q}(s, a)$ and its visiting count $N(s, a)$ are initialized to 0 and will be updated in the Backpropagation stage. $N_{\text{parent}}(a)$ represents the visiting count of the parent node of a . The prior $\pi(a|s_t)$ is the exponential of the averaged log-probability of all tokens in the step a , i.e., $\exp\left(\frac{1}{|a|} \sum \log \pi(a_j|a_{<j}, s_t)\right)$. c_{puct} is a weight control parameter and we use its default value 0.5. Therefore, the node with high log-probability and low visiting counts will be prioritized. The action selection iterates until it encounters a leaf node of the current search tree.

Expansion. In this stage, we expand the node by generating the next step. Specifically, we first trace the selected leaf node to the root and obtain a partial solution $s_t = \{s_0, a_1, \dots, a_t\}$. The partial solution contains the problem description and current generated steps and serves for further node expansion. Different from traditional reinforcement learning scenario where actions are always limited [46], given that the LLM can theoretically generate a nearly unlimited number of potential reasoning steps [21], we employ sampling-based generation to obtain promising and diverse steps. To accurately identify the end of each step generation, we utilize in-context learning with XML-style like delimiters “<step>” and “</step>” to wrap each step. The LLM output is truncated after the closing delimiter “</step>”.

Evaluation. In this stage, we evaluate the quality of the newly expanded node s_t . Specifically, if it reaches a leaf node that contains the final solution code, we evaluate its correctness using test cases u and compute its reward according to the following function:

$$\hat{V}(s_t) = r = \begin{cases} 1, & \text{if all test cases pass} \\ -1, & \text{if any test case fails} \end{cases} \quad (2)$$

For non-terminal expanded nodes, we set the value to 0 since there are no clear signals indicating their correctness at intermediate stages.

Backpropagation. At the end of each iteration, we update each node (s) along the path from the leaf node s_t to the root s_0 . Following [7, 46], we update their visiting counts and state-action values according to the following rules: $N(s, a) \leftarrow N(s, a) + 1$ and $\hat{Q}(s, a) \leftarrow \frac{1}{N(s, a)} \sum_{j=1}^N \mathbb{I}_{s, a \rightarrow s_j} \hat{V}(s_j)^{(j)}$. The $\hat{Q}(s, a)$ measures the probability of generating correct code when taking action a at state s , i.e., the value represents the probability of generating a correct solution when continuing from the current path.

The above four stages will continue until reach the maximum iteration numbers. After that we obtain the final tree $\mathcal{T}$ for each problem, which stores the expanded nodes and their corresponding state-action values $Q(s, a)$.

3.1.2 Path Perturbation and Refinement. Despite leveraging MCTS, the constructed trees do not always include both correct and incorrect paths for all programming problems. For example, for DeepSeek-Coder-6.7B-Instruct, 29.5% samples during the collection process contain only correct paths as the problems might be too simple for the model to solve, while 11.8% samples contain only incorrect paths due to the model’s limited capabilities. The lack of correct reasoning training data can limit the policy model’s generation capabilities, while a lack of incorrect paths can impair the value model’s ability to assess the quality of intermediate steps. To address this issue, we propose path perturbation and refinement methods to enrich the reasoning data for problems that include only correct or incorrect paths, with details as below.

(1) *For samples including only correct reasoning paths*, we propose path perturbation to create incorrect paths for them. This approach

improves the model's quality assessment ability for basic programming problems where MCTS fails to identify incorrect reasoning paths. Specifically, we first randomly partition the original correct reasoning path into two segments: $s_0, a_1, \dots, a_t$ and $a_{t+1}, \dots, a_T$. Next, we retain only the first segment while perturbing the problem description at the root node by preserving just the initial sentence. Given this ambiguous requirement and the partial reasoning steps, LLMs generate subsequent steps and code that do not well fit the original requirement. Finally, after verifying through test cases that the generated code is incorrect, we replace the ambiguous requirement with the original specification to create an incorrect reasoning path. We further provide an example in our replicate package for illustration [26]. This process simulates scenarios where the model hallucinates or misinterprets requirements, which is a common occurrence in LLM-generated code [67].

(2) For samples including only incorrect reasoning paths, we propose path refinement to synthesize correct reasoning paths for them. Specifically, we propose to leverage the ground truth solutions to refine the reasoning process through self-reflection [38]. As shown in Figure 4, using a reflection prompt template (available in the GitHub repository [26]), we present the failed reasoning paths alongside the ground truth solution, guiding the model to identify specific errors and generate refined reasoning paths.

After these steps, we construct our reasoning dataset for training D_{train} by sampling paths from the MCTS trees $\mathcal{T}$, along with new paths generated through path perturbation and refinement processes. For the actions, we calculate their values differently based on the path source. For paths from MCTS and path perturbation, we obtain values from their respective MCTS trees. Specifically, according to the transition function $Q(s_t, a_t) = r(s_t, a_t) + V(s_t) = V(s_{t+1})$ for non-terminal nodes [7, 46], we could employ the $Q(s_t, a_t)$ as the value v_t of action a_t . For paths obtained through path refinement, we assign a value of 1 to the final state. Finally, each sample in D_{train} consists of a problem description q paired with a sequence of reasoning steps $(a_1, v_1), \dots, (a_n, v_n)$, where a_i and v_i represent the content of the step and its associated value, respectively.

3.2 Reasoning Quality-aware Model Training

This part aims at training LLMs to generate high-quality reasoning steps and select the best generated step. Following the practice in AlphaZero [47], we propose a dual reasoning system that contains a policy model and a value model. We employ the pre-trained LLM as the policy model π_θ . In this work, we specifically use fine-tuned DeepSeek-Coder-6.7B-Instruct and Qwen2.5-Coder-7B-Instruct as our policy models. To save cost, we do not introduce another model as the value model V_ϕ but extend the policy model by only adding an auxiliary linear layer with the tanh activation function [32], which works alongside the traditional softmax layer responsible for token prediction, as shown in Figure 4 (2). This design implies that these two models, π_θ and V_ϕ , share most of their parameters. Based on the correct paths X^+ and incorrect paths X^- from our constructed dataset D_{train} , we train the policy and value models through multi-task learning as follows:

$$\arg \min_{\theta, \phi} \sum_{\mathbf{x} \in X^+} -\log \pi_\theta(\mathbf{x}|\mathbf{q}) + \beta \sum_{\mathbf{x} \in X^+ \cup X^-} \sum_{t=1}^{T(\mathbf{x})} |V_\phi(s_t) - v_t|^2 \quad (3)$$

Algorithm 1 Adaptive reasoning step search algorithm

Input: Beam sizes B_1, B_2 , question s_0 , max steps T .

Output: Top-1 results

```

1:  $C \leftarrow [s_0] \times B_1, t \leftarrow 0$ 
2: while  $t < T$  and non-terminal path in  $C$  do
3:   Initialize priority queue  $C_{t+1}$ 
4:   for  $s_t \in C$  do
5:     if  $t=0$  then
6:       Sample  $\{a^{(b)}\}_{b=1}^{\lceil B_2/2 \rceil} \sim \pi_\theta(a | s_t)$ 
7:       Sample  $\{a^{(b)}\}_{b=1}^{\lceil B_2/2 \rceil} \sim \pi_\theta(a | s_t, \dots, \{\text{lang}\})$ 
8:     else
9:       Sample  $\{a^{(b)}\}_{b=1}^{B_2} \sim \pi_\theta(a | s_t)$ 
10:    end if
11:  end for
12:  for  $b = 1$  to  $B_2$  do
13:     $s_{t+1} \leftarrow \text{Concat}[s_t, a^{(b)}]$ 
14:    Add  $(s_{t+1}, V_\phi(s_{t+1}))$  to  $C_{t+1}$ 
15:  end for
16:   $C \leftarrow \text{Top-}B_1 \text{ of } C_{t+1}$ 
17: end while
18: return Top-1 of  $C$ 

```

where $\mathbf{x} = \{a_1, a_1, \dots, a_n\}$. The first term represents the negative log-likelihood loss for reinforced fine-tuning the policy model on correct paths, and the second term captures the loss in value prediction for both correct and incorrect solutions, respectively. $T(\mathbf{x})$ denotes the number of steps for path $\mathbf{x}$. β is a tunable hyper-parameter to control the weight of value model loss.

3.3 Adaptive CoT Reasoning

This stage aims at teaching models to apply chain-of-thought reasoning adaptively. Based on the step evaluation capability of the value model, we propose extending it to judge whether to use CoT reasoning for different problems. To this end, we first collect training data containing directly-generated solutions and their value scores by the models themselves, then continually fine-tune the value model on this data. Specifically, as shown in Figure 4 (3), we feed the LLM with the seed data D_{seed} and force the model to directly output the code without CoT reasoning. We then evaluate these directly-generated outputs using test cases u . By assigning the value of correct solutions as 1 and incorrect solutions as -1, we build the adaptive reasoning dataset D_{ar} that contains the question, code, and value tuple (q, a, v) . To prevent catastrophic forgetting during continual training [20, 39], we incorporate a KL divergence regularization term on the policy model's predictions. In addition, to maintain balance between correct and incorrect samples, we perform downsampling for the combined dataset, i.e., D_{train} and D_{ar} , and train the model as follows:

$$\arg \min_{\theta, \phi} \sum_{\mathbf{x} \in X^+} KL(\pi_\theta(\mathbf{x}|\mathbf{q}), \pi_{old}(\mathbf{x}|\mathbf{q})) + \beta \sum_{\mathbf{x} \in X^+ \cup X^-} \sum_{t=1}^{T(\mathbf{x})} |V_\phi(s_t) - v_t|^2 \quad (4)$$

where X^+ and X^- denotes the correct paths and incorrect paths from the concatenation dataset, respectively. π_{old} is the model obtained in Section 3.2.

During model inference, we propose an adaptive reasoning step search algorithm by employing the trained policy and value model, as detailed in Algorithm 1. The overall procedure follows traditional beam search while using the value model to guide the decoding process. It contains two beam size parameters: B_1 which represents the number of preserved current best samples (similar to traditional beam search), and B_2 which controls the number of candidate actions generated by the policy model at each step. The process operates as follows: In the first iteration ($t = 0$), the policy model generates $\lfloor B_2/2 \rfloor$ reasoning steps based on the programming problem s_0 , and $\lfloor B_2/2 \rfloor$ solutions without reasoning. To force the model to generate solutions without reasoning, the response prefix ``{lang}`` is appended to the end of the prompt, where lang is the target programming language (lines 5–7). Then, the algorithm leverages the value predictions from the value model V_ϕ to rank the steps and directly-generated solution, determining whether to use step-by-step reasoning or not. The newly expanded states s_{t+1} and predicted value are stored in a priority queue C_{t+1} to enable efficient beam search (lines 12–14). From the second iteration onward, the policy model generates B_2 reasoning steps based on current paths in C (lines 8–9) and the value model assesses the quality of each reasoning path as in the first iteration. Finally, this process continues until either the maximum number of steps T is reached or all paths are terminated. At the end of the search, the top candidate is returned as the final answer (line 18).

4 EXPERIMENTAL setup

4.1 Research Questions

In the evaluation, we focus on the following four research questions:

- RQ1:** How effective is SEER in improving chain-of-thought code generation?
- RQ2:** What are the contributions of different modules in SEER?
- RQ3:** What is the impact of different hyper-parameters on the performance of SEER?

To study RQ1, we conduct a comprehensive evaluation of SEER on three widely used code generation benchmarks and compare it with four representative baseline methods across two state-of-the-art open-source code LLMs. For RQ2, we remove different parts in SEER to assess their individual contributions. For RQ3, we perform a detailed analysis of hyperparameters in SEER.

4.2 Benchmarks and Metrics

Following previous work [50, 54], we evaluate performance on three widely-used coding benchmarks: MBPP [2], HumanEval [8], and LiveCodeBench [27].

- MBPP [2] consists of 378 crowd-sourced Python programming problems, which is hand-verified by the original authors. We follow previous work [50] and report results on both the original and plus versions, which include more test cases [35].
- HumanEval [8] consists of 164 manually-written Python programming problems, each accompanied by an average

of 9.6 test cases. Following previous work, we report the performance of both original and plus versions [35].

- LiveCodeBench (LCB) [27] is a contamination-free programming benchmark that collects new problems over time from contest platforms. We include all available 713 programming problems during the experimental period.

To measure the accuracy of the predicted code, we use the widely-adopted pass rate metric (pass@1) [37, 50, 54].

4.3 Models and Baselines

In this paper, we select two popular and state-of-the-art code LLMs for experiments:

- **DeepSeek-Coder (DSC)** is trained from 2T tokens from scratch, achieving state-of-the-art performance on a variety of code intelligence tasks. Specifically, we choose DeepSeek-Coder-6.7B-Instruct in this paper.
- **Qwen2.5-Coder (QC)** is a family of LLMs based on models of the Qwen2.5 series and continues to be pretrained on a vast corpus of over 5.5 trillion tokens, presenting state-of-the-art performance in both code generation and reasoning. We choose the 7B version of Qwen2.5-Coder-Instruct.

We compare our method with recent representative COT-based code generation methods from top venues, including two representative prompting-based COT code generation methods, Self-planning and Code COT, and a training-based SFT method used in [60]. Besides, considering the popularity of DeepSeek-R1, we further include its training method as a baseline.

- **Self-Planning** [30] first leverage LLMs to plan concise solution steps from the intent combined with a few-shot prompt and then generate code step by step, guided by the preceding solution steps. We replicate it based on the few-shot examples provided in their paper.
- **Code CoT** [30] is a prompting method that asks LLMs to describe the implementation code line by line and then write the code. We also replicate it based on the prompt provided in the paper.
- **SFT** [60] fine-tunes LLMs with data that contain chain-of-thought prompts and final solution code. Specifically, we leverage the data provided by COTTON [60] which is crafted by distilling CoTs from GPT-4.
- **DS-R1's RL** [21] is a reinforcement learning method used to train DeepSeek-R1-Zero. It is the state-of-the-art LLM reasoning enhancement method. We replicate it based on the code provided by the Open-R1 repository [40]. We train the model with the same seed data used in SEER and employ the same reward functions as used in DeepSeek-R1-Zero, namely the format reward and the accuracy reward.

We also compare SEER with other open-source models with much larger parameters such as StarCoder2-15B-Instruct-v0.1 [36], DeepSeek-Coder-33B-Instruct [22], Llama3-70B-Instruct [13], Mixtral-8x22B-Instruct-v0.1 [29], and DeepSeek-Coder-V2-Instruct [69]. We also compare with some closed-source proprietary models including GPT-4o, GPT-4o-mini, GPT-4 [41] and Gemini 1.5 Flash 002 [1].

Table 1: Comparison results with baseline models. The bold figures indicate the best results.

Approach		Size/Version	MBPP		HumanEval		LiveCodeBench Avg
			Original	Plus	Original	Plus	
Closed-source Proprietary Models							
GPT-4o		Nov. 2024	87.0	71.4	92.1	87.2	45.0
GPT-4o-mini		July. 2024	85.4	72.2	88.4	83.5	39.4
GPT-4-Turbo		Apr. 2024	85.7	73.3	90.2	86.6	44.2
Gemini 1.5 Flash 002		Sept. 2024	68.0	59.0	82.3	75.6	36.2
15B+ Models							
DeepSeek-Coder-V2-Instruct		236B	89.4	75.1	85.4	82.3	41.9
Mixtral-8x22B-Instruct-v0.1		176B	73.8	64.3	76.2	72.0	24.2
Llama3-70B-Instruct		70B	82.3	69.0	77.4	72.0	26.2
DeepSeek-Coder-33B-Instruct		33B	80.4	70.1	81.1	75.0	23.4
StarCoder2-15B-Instruct-v0.1		15B	75.2	61.2	67.7	60.4	14.6
7B-level Models							
DeepSeek-Coder-6.7B-Instruct	Direct Generation	6.7B	72.7	63.4	73.8	70.1	18.5
	Self Planning	6.7B	74.1	62.4	77.4	71.3	18.7
	Code CoT	6.7B	73.8	60.5	74.4	65.9	18.2
	SFT	6.7B	72.2	61.9	73.2	65.9	18.1
	DS-R1's RL	6.7B	74.3	65.1	77.4	71.3	20.1
	SEER	6.7B	82.0	72.2	82.9	73.8	23.8
Qwen2.5-Coder-7B-Instruct	Direct Generation	7B	83.5	71.7	88.4	84.1	33.2
	Self Planning	7B	84.1	72.0	84.1	78.0	29.2
	Code CoT	7B	81.2	69.6	81.1	74.4	28.1
	SFT	7B	82.0	71.7	86.0	81.1	28.1
	DS-R1's RL	7B	84.7	71.7	89.0	84.8	34.1
	SEER	7B	88.6	75.1	90.9	86.0	36.7

4.4 Implementation details

We employ Self-OSS [53] as our seed dataset for two key reasons: first, its correctness can be reliably assessed through unit tests, and second, the authors of Self-OSS have conducted a thorough data decontamination process [53] to remove questions that overlap with popular benchmarks. To save experimental costs, we randomly select 10K question-code pairs from Self-OSS for MCTS data collection. During the expansion stage of MCTS, we employ in-context learning and incorporate three demonstrations which are randomly selected from a pool of eight prepared examples. These eight examples align with previous few-shot CoT code generation methods [30]. The detailed prompt templates used in this paper can be found in the GitHub repository [26]. We construct one search tree per sample in the seed data, with an average of 254 and 261 nodes per tree for Deepseek-Coder and Qwen2.5-coder, respectively. When extracting paths from MCTS tree to construct D_{train} , we first filter paths with incorrect formats and randomly sample up to three correct and three incorrect reasoning path for each problem. We then perform downsampling to keep a balance between positive and negative samples. This yields finetuning datasets of 68K and 60K samples, respectively. Similarly, we also conduct the same data sampling process for data generated in the path perturbation and refinement process. The initial data construction takes 43 and 47 hours respectively, but this process is performed only once and the data can be reused.

Following previous studies [54], we train the model for two epochs with a batch size of 128. During inference, we set the value loss weight β to 0.025, the LLM temperature to 0.5, and configure

B_1 and B_2 to 3 and 4, respectively. The influence of these parameters is discussed in Section 5.3. Finetuning with data from MCTS requires approximately 12 hours and 10 hours on one GPU for these two models, which is similar to previous work [54] when using the same training data size and epochs. We repeat SEER three times and report its average results to eliminate the influence of sampling and fluctuations in LLM. During adaptive CoT reasoning training, incorporating KL divergence increases GPU memory requirements. Due to our server's memory limitations, we employ the parameter-efficient tuning [23, 48] method LoRA [24] to prevent out-of-memory errors. For Qwen, we observed that the model already generates high-quality Chain-of-Thought reasoning with prompting alone, and additional fine-tuning might degrade performance. Therefore, we apply KL divergence during the first training stage for Qwen as well. All experiments are conducted on a server running Ubuntu 20.04 with 4 NVIDIA Tesla A100 GPUs (40GB memory each).

5 Experiments

5.1 RQ1: Performance Evaluation

To evaluate the effectiveness of SEER in improving the reasoning capabilities of code LLMs, we compare it with multiple baselines across three different evaluation benchmarks. Table 1 presents the performance of SEER and baseline methods on MBPP, HumanEval, and LiveCodeBench. Based on these results, we derive the following findings:

Simple prompting and SFT provide limited improvements in LLMs’ reasoning ability for code generation. As shown in Table 1, we observe that simple prompting and SFT have limited effectiveness in enhancing model reasoning capabilities. For instance, DeepSeek-Coder-6.7B-Instruct with Self-Planning prompt shows only a marginal improvement on HumanEval Original from 73.8% to 77.4%, while experiencing a decrease on MBPP Plus from 63.4% to 62.4%. Similarly, Qwen2.5-Coder-7B-Instruct with SFT exhibits a reduction in performance on the MBPP Original benchmark, declining from 83.5% to 82.0% compared to the base model. These results suggest that merely applying prompt engineering or supervised fine-tuning fails to effectively improve reasoning ability for complex code generation tasks.

SEER effectively improve models’ reasoning capabilities in code generation. As shown in Table 1, models augmented with SEER demonstrate substantial improvements across all evaluation benchmarks. For DeepSeek-Coder-6.7B-Instruct, using SEER improves performance by 9.3%, 9.1% and 5.3% in terms of pass@1 on MBPP Original, HumanEval Original and LiveCodeBench, respectively. Notably, SEER outperforms the state-of-the-art baseline DS-R1’s RL by 7.7%, 5.5%, and 3.7% on these same benchmarks. Similarly, Qwen2.5-Coder-7B-Instruct enhanced with SEER shows remarkable gains, improving by 5.1%, 2.5% and 3.5% on MBPP Original, HumanEval Original and LiveCodeBench compared to the base model, while surpassing DS-R1’s RL by 3.9%, 1.9%, and 2.6% respectively. These consistent improvements across multiple benchmarks and the clear advantage over other reasoning enhancement techniques like DS-R1’s RL demonstrate SEER’s effectiveness in augmenting model reasoning capabilities for code generation tasks.

Small models augmented with SEER outperform some larger models. By comparing the performance of models of different sizes, we find that smaller models enhanced with SEER can outperform certain larger models. For example, Qwen2.5-Coder-7B-Instruct+SEER achieves 88.6% and 75.1% on MBPP Original and Plus, outperforming GPT-4o, which scores 87.0% and 71.4%, respectively. Similarly, DeepSeek-Coder-6.7B-Instruct+SEER attains 82.9% on HumanEval Original, exceeding both Llama3-70B-Instruct (77.4%) and DeepSeek-Coder-33B-Instruct (81.1%) with 4-10 times more parameters. These results highlight that through SEER, smaller models can achieve comparable or superior performance to larger models in code generation.

Answer to RQ1: SEER is highly effective in improving CoT-based code generation, consistently improving performance across all model sizes and enabling smaller models to outperform much larger ones.

5.2 RQ2: Ablation Study

5.2.1 Impact of different components. In this section, we validate the contribution of different components of SEER, including the path perturbation and refinement in diverse reasoning path exploration, the value model-based reasoning quality assessment, and adaptive CoT reasoning.

Path perturbation and path refinement. To assess the impact of our path exploration strategies, we independently remove them from the path exploration process. Path refinement augments 3,051 and 3,499 correct reasoning paths for DeepSeek-Coder and

Table 2: Ablation study of different components in SEER. The bold figures indicate the best results, where PP and PR denote path perturbation and path refinement, respectively.

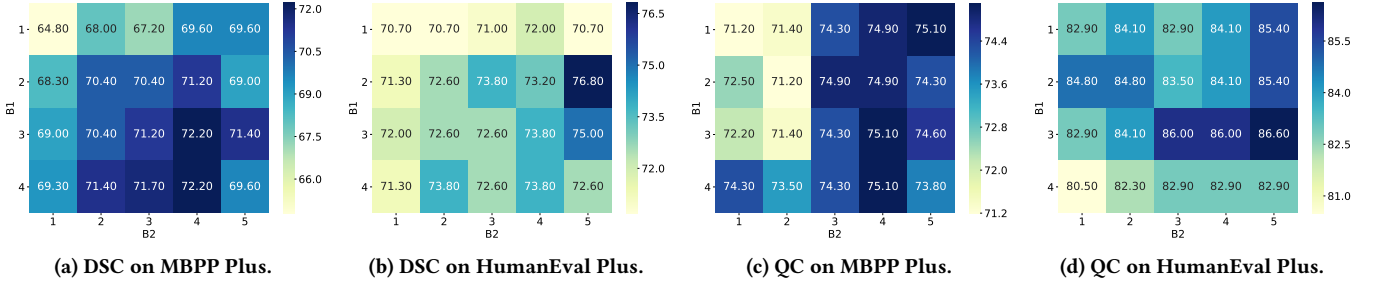
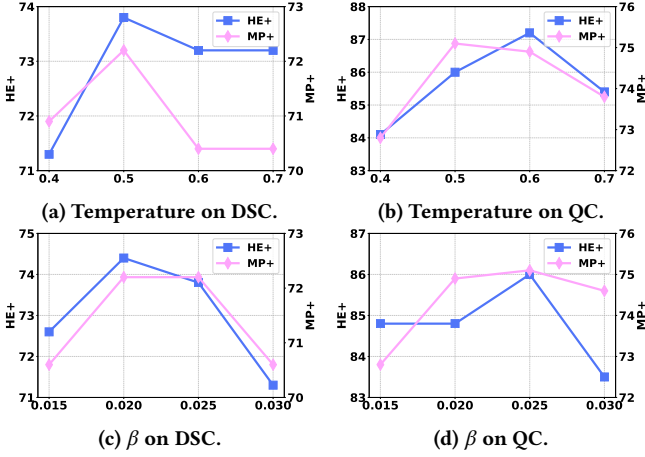
Approach	MBPP		HumanEval		LCB Avg
	Original	Plus	Original	Plus	
DeepSeek-Coder-6.7B-Instruct					
SEER	82.0±1.1	72.2±0.6	82.9±1.0	73.8±1.2	23.8±0.5
-w/o PP	81.2±0.4	70.2±1.9	80.7±1.8	70.7±0.6	23.5±0.3
-w/o PR	80.9±0.9	69.1±1.2	80.3±1.5	74.6±0.7	23.3±0.6
-w/o value model	76.6±0.8	64.6±0.3	77.3±1.3	69.9±1.4	18.6±0.4
-w/o adaptive cot	81.0±0.9	69.0±1.7	79.3±1.1	70.3±2.2	21.9±1.4
Qwen2.5-Coder-7B-Instruct					
SEER	88.6±0.9	75.1±0.3	90.9±0.0	86.0±2.2	36.7±0.3
-w/o PP	87.3±1.0	73.5±1.1	89.8±0.3	84.6±1.0	37.0±0.4
-w/o PR	87.0±0.7	73.6±0.6	90.0±0.8	85.8±1.2	35.9±0.4
-w/o value model	84.1±0.9	71.6±0.5	88.8±0.3	83.3±1.3	31.3±0.6
-w/o adaptive cot	86.4±0.8	73.4±1.2	87.7±0.8	83.9±0.8	35.2±0.6

Table 3: The impact of different training strategies. The bold figures indicate the best results.

Approach	MBPP		HumanEval		LCB Avg
	Original	Plus	Original	Plus	
DeepSeek-Coder-6.7B-Instruct					
SEER	82.0±1.1	72.2±0.6	82.9±1.0	73.8±1.2	23.8±0.5
Only first stage	81.0±2.0	70.4±1.8	80.4±0.4	71.8±2.5	18.7±0.9
Two stage no KL	77.8±0.6	67.5±0.3	76.0±2.1	67.6±0.8	19.5±1.1
Qwen2.5-Coder-7B-Instruct					
SEER	88.6±0.9	75.1±0.3	90.9±0.0	86.0±2.2	36.7±0.3
Only first stage	86.8±1.6	73.4±1.2	88.0±0.9	83.3±1.3	34.4±0.2
Two stage no KL	74.3±0.7	64.5±0.9	72.2±0.3	67.1±1.2	23.6±0.5

Qwen2.5-Coder, respectively, while path perturbation provides 7,834 and 6,169 negative training samples for these models. As shown in Table 2, removing path perturbation leads to obvious performance degradation on two relatively basic datasets MBPP Plus and HumanEval Plus. For example, Deepseek-Coder’s performance on HumanEval Plus drops from 73.8% to 70.7% when this component is excluded. This shows that path perturbation could enhance the model’s quality assessment on basic programming problems. Similarly, removing path refinement leads to a larger degradation, particularly on MBPP Plus where DeepSeek-Coder experiences a 3.1% decrease. These results demonstrate that both path exploration strategies play crucial roles in improving model generalization across different scenarios by enhancing the diversity of collected paths.

Value model-based reasoning quality assessment. We further investigate the importance of the value model-based reasoning quality assessment by removing the validation of intermediate reasoning steps, i.e., changing $V_\phi(\mathbf{s}_{t+1})$ in line 14 in Algorithm 1 to 0 and selecting C randomly in line 16. This leads to tremendous performance drops, with DeepSeek’s performance on LiveCodeBench decreasing by 5.2% and Qwen2.5’s by 5.4%. This demonstrates the critical importance of validating the quality of intermediate reasoning steps during inference. We provide a case in Section 6.1 for further illustration.

Figure 5: Parameter analysis of B_1 and B_2 .Figure 6: Parameter analysis of temperature and value model weight β .

Adaptive CoT reasoning. Finally, we validate the effectiveness of the adaptive CoT reasoning mechanism by disabling direct solution generation in line 7 in Algorithm 1. The performance decreases are substantial, with DeepSeek-Coder showing a 3.6% drop on HumanEval Original and a 3.2% drop on MBPP Plus. For Qwen2.5-Coder, we observe a 3.2% reduction on HumanEval Original and a 1.7% decrease on MBPP Plus. Furthermore, removing adaptive CoT reasoning also increases average code generation time. For example, on MBPP, SEER enables to only apply step-by-step reasoning for 65.6% and 73.5% of questions for DeepSeek-Coder and Qwen2.5-Coder, respectively. Removing this component increases generation time for these two models by 34.5% and 32.6%. These results indicate that adaptive CoT reasoning effectively contributes to choosing the appropriate generation approach and reducing unnecessary computational cost.

5.2.2 Impact of different training strategy. In this section, we validate the effectiveness of our model training strategy. SEER consists of two training stages: (1) an initial phase using data collected during diverse reasoning path exploration, followed by (2) adaptive CoT training with modified objectives. To validate this design, we construct two variants of our training strategy, including using only the first phase without the adaptive CoT training and maintaining the two-phase approach but without the KL divergence modification in the second phase.

Only First Stage. When limiting SEER to only the first stage of training, we observe performance degradation across all benchmarks. For example, with DeepSeek-Coder, this variant achieves 80.4% on HumanEval Original and 71.8% on HumanEval Plus, representing a notable decline compared to the SEER. This suggests that the absence of adaptive CoT reasoning training impairs the model’s ability to assess reasoning quality, thus hampering its capacity to effectively select between step-by-step reasoning and direct generation approaches.

Two Stage no KL. The two-stage approach without object modification shows remarkable performance degradation across all tested models. For example, on LiveCodeBench, this variant performs 4.3% worse than SEER with DeepSeek-Coder and 13.1% worse with Qwen. These results demonstrate that without KL regularization, models suffer from severe catastrophic forgetting, degrading their original generation capabilities.

Answer to RQ2: All components contribute to SEER for code generation. Our two-stage training strategy demonstrates the best performance among different variants.

5.3 RQ3: Parameter Analysis

In this section, we study the impact of four parameters on the performance of SEER, including the LLM temperature, the weight of value model β in Equ. 3, and the beam search width B_1 and B_2 in the Algorithm 1. Due to space limitation, we only present the results on HumanEval Plus and MBPP Plus, with results for other benchmarks presented on our GitHub repository [26].

Temperature. To investigate the impact of temperature on model performance, we vary the temperature from 0.4 to 0.7 and display the results in Figures 6 (a) and (b). For DeepSeek-Coder, performance on both HumanEval Plus and MBPP Plus datasets achieves optimal results when the temperature is set to 0.5. In the Qwen2.5-Coder evaluation, the performance on HumanEval Plus reaches its peak at a temperature of 0.6, while MBPP Plus performs best at 0.5. In general, either lower (0.4) or higher (0.7) temperature values cannot yield better results. Based on these findings, we set the temperature parameter to 0.5 to obtain relatively stable and excellent performance across different models and datasets.

Value model weight β . We study the effect of β by adjusting its value from 0.015 to 0.030, as shown in Figure 6 (c) and (d). For DeepSeek-Coder, performance on HumanEval Plus peaks at a weight of 0.020, while performance on MBPP Plus performs best at 0.025. For Qwen2.5-Coder, performance on both HumanEval Plus and MBPP Plus datasets achieves optimal results at 0.025. When

the weight is too small, the value model may not learn effectively to assess the quality of the reasoning paths. In contrast, when the weight increases to 0.030, performance decreases on both datasets across both evaluation models, suggesting that excessively high weight values may impair the generation capabilities of the policy model. Therefore, we chose to set the weight parameter to 0.025 to achieve balanced performance across different encoder models and evaluation datasets.

Beam size B_1 and B_2 . Figure 5 presents the influence of B_1 and B_2 on model performance. We systematically increase the values of both parameters from (1,1) and monitor the resulting performance. For DeepSeek-Coder, we observe a gradual improvement in performance on HumanEval Plus as shown in Figure 5 (a), with performance increasing until approximately $B_1=3$ and $B_2=4$. A similar trend is evident in Figure 5 (b) for DSC on MBPP Plus, where the optimal values cluster around the similar region. For Qwen2.5-Coder, Figures 5 (c) and (d) demonstrate comparable behavior, with performance metrics stabilizing around $B_1=3$ and $B_2=4$ for both HumanEval Plus and MBPP Plus datasets. Although further increasing these parameters might yield marginal improvements, the additional computational cost would not justify such minimal gains. Therefore, we select $B_1=3$ and $B_2=4$ as our final parameter values.

Answer to RQ3: The LLM temperature and value model weight β could affect the accuracy of SEER, with intermediate values yielding optimal performance. Increasing beam sizes B_1 and B_2 brings substantial improvement, though gains become relatively marginal beyond $B_1=3$ and $B_2=4$.

6 Discussion

6.1 Why SEER works for code generation?

To better understand how SEER helps CoT reasoning for code generation, we present an example from the MBPP benchmark in Figure 7. As demonstrated, the baseline DeepSeek-Coder directly generates a solution by using a single `replace()` function, which fails to handle the bidirectional replacement requirement in the test case “The_Advanger”. This illustrates a fundamental limitation of direct generation approaches, where models often lack comprehensive requirement analysis and overlook complex edge cases. In contrast, our SEER approach allows DeepSeek-Coder to break down the problem through step-by-step reasoning, leading to a character-by-character iteration solution that correctly handles both space-to-underscore and underscore-to-space conversions. However, when the value model-based reasoning quality assessment is removed and the traditional beam search is used instead, the model produces flawed reasoning from the very first step by attempting to use the `replace()` function. Although this solution is closer to the correct answer than the baseline, its reasoning process overlooks the coverage problem of two consecutive transformations. This indicates that value model-based reasoning quality assessment plays a crucial role in guiding the reasoning process.

6.2 Relationship with Large Reasoning Models

Recently, Large Reasoning Models like OpenAI-O3 [42] and DeepSeek-R1 [21] have gained attention for using “long CoT” to achieve test-time scaling [57, 58, 66]. Unlike traditional CoT’s linear, step-by-step generation process, long CoT allows models to

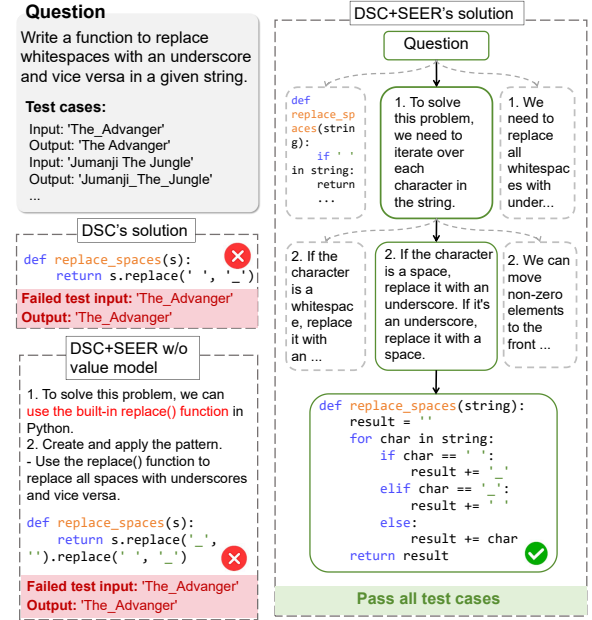


Figure 7: Case studies of comparing SEER with baseline methods. The dashed lines in represent unselected steps.

iteratively refine their thoughts through activities like reflection and backtracking [21]. SEER is a form of test-time scaling based on tree search, which is orthogonal to the long CoT-based method. Tree search methods generate multiple candidate steps and use value models to select the most promising ones, ensuring step-wise reliability [58]. In contrast, long CoT employs linear reasoning to continuously revise content, offering broader domain applicability without requiring auxiliary scoring models. Consequently, long CoT can suffer from “overthinking” and inefficiency issue due to extended generation times [5, 58]. Meanwhile, recent research [9, 61] also indicates that these two approaches can be further combined to achieve better performance. For example, [9] has shown that LRMs naturally use certain symbols like “Wait” to distinguish different steps, so these multiple steps can be viewed as a decision-making process where MCTS can be used to construct step-by-step process supervision data. Our proposed method can then be employed to enrich the diversity of training data and assist efficient model reasoning.

We further conduct experiments to compare SEER with a representative open-source LRMs R1-Distill-Qwen-7B [21], with respect to both accuracy and generation time. Comparing DeepSeek-R1-Distill-Qwen-7B with SEER is not entirely fair, as it requires data distilled from a proprietary model, whereas SEER does not. However, the experimental results demonstrate that our method achieves comparable overall performance. Specifically, our approach outperforms DeepSeek-R1-Distill-Qwen-7B on MBPP with a pass rate by 7.4%. More importantly, our method consistently requires significantly less inference time across all benchmarks. Based on the average results across various datasets, our method achieves 75.6% compared to 73.0% for DeepSeek-R1-Distill-Qwen-7B, while maintaining time efficiency advantages. This demonstrates the effectiveness of SEER.

Table 4: Comparison with DeepSeek-R1-Distill-Qwen-7B.

Methods	MBPP			HumanEval			LiveCodeBench		Average
	Pass rate	Plus	time	Pass rate	Plus	time	Pass rate	time	
Qwen2.5-Coder-7B-Instruct+SEER	88.6	75.1	7min40s	90.9	86.0	5min20s	36.7	73min	75.6
DeepSeek-R1-Distill-Qwen-7B	81.2	67.7	12min49s	91.5	84.1	6min09s	40.5	99min	73.0

Table 5: Results of combining SEER and Reflexion.

Approach	MBPP		HumanEval		LCB Avg
	Original	Plus	Original	Plus	
Deepseek-Coder-6.7B					
SEER	82.0	72.2	82.9	73.8	23.8
SEER +Reflexion	83.9	75.7	82.9	74.4	25.7
Qwen2.5-Coder-7B-Instruct					
SEER	88.6	75.1	90.9	86.0	36.7
SEER +Reflexion	88.9	77.0	91.5	86.6	38.3

6.3 Relationship with Reflexion

In this section, we further validate whether SEER could be combined with existing reasoning frameworks like Reflexion [45]. Reflexion is a framework that iteratively refines code generation through self-reflection and error correction. SEER aims to enhance code generation accuracy in the first attempt while Reflexion uses iterative generation and validation which is orthogonal to our method. The experimental results in Table 5 demonstrate that combining SEER with Reflexion consistently improves performance across all evaluated benchmarks and model sizes. For the Deepseek-Coder, SEER +Reflexion achieves notable improvements on MBPP Plus with 3.5 percentage points gain from 72.2% to 75.7% and on LCB average with 1.9 percentage points improvement from 23.8% to 25.7%. Similarly, for the Qwen2.5-Coder, SEER +Reflexion demonstrates consistent gains across all benchmarks, with improvements ranging from 0.6 to 1.6 percentage points. These results indicate the complementary benefits of our approach with existing self-reflection techniques for code generation.

6.4 Threats to Validity

We have identified the following major threats to validity:

- (1) *Limited LLMs.* In this work, we select two state-of-the-art pre-trained models Deepseek-Coder and Qwen2.5-Coder for evaluation. To comprehensively evaluate the performance of SEER, we will validate whether SEER is also effective on other models such as Llama [13] in the future.
- (2) *Potential data contamination.* Another threat is potential data contamination. To address this, we utilize the Self-OSS dataset as our seed data, where the original authors have already performed comprehensive data decontamination and removed benchmark questions with high similarity. Additionally, we conduct experiments on LiveCodeBench, a contamination-free benchmark that collects new problems over time.

7 Related work

7.1 Code Generation

Code generation aims to automatically create software code based on user intent. Recent advances in deep learning techniques, particularly LLMs, have significantly advanced this field. Several code LLMs trained on massive code datasets have achieved promising results, including StarCoder [36], Code Llama [44], DeepSeek-Coder [22] and Qwen2.5-coder [25]. Apart from training the code LLM, recent work has also focused on improving code generation performance through iterative generation and fixing approaches. For example, Dong et al. [12] propose a self-collaboration method that enhances LLMs’ performance by employing multiple LLMs as different roles in software development. Self-edit [65] utilizes compiler error messages to enhance the correctness of code generation. Our approach focuses on training the code LLMs to improve its reasoning ability and enhance code generation accuracy on the first attempt, which could complement these methods to further improve their performance.

7.2 LLM Reasoning

Large language models (LLMs) have demonstrated the emergent ability to generate intermediate reasoning steps for complex problem-solving which is known as chain-of-thought reasoning [31, 52]. To leverage this capability, researchers have proposed various prompting methods including Least-to-Most prompting [68], Program-of-Thoughts (PoT) [10], Self-Planning [30], and Self-Refine [38]. Recent studies have also explored enhancing LLM reasoning through supervised fine-tuning [62, 63]. Due to the high cost of manual data annotation, researchers have developed methods that synthesize reasoning process data by LLMs. For instance, the Self-Taught Reasoner (STaR) [62] generates rationales and fine-tunes on those that lead to correct answers. For complex multi-step reasoning tasks like mathematical and commonsense reasoning, some approaches employ MCTS to collect process supervision data for model training [7, 59, 63]. Different from those work, our work present as the first work to improve LLM’s reasoning ability for code generation and improves the MCTS-based data collection through path perturbation and refinement to increase data diversity.

8 Conclusion

In this paper, we introduce SEER, a novel framework that formulates code generation as a decision-making problem with diverse reasoning path exploration, reasoning quality-aware model training, and adaptive CoT reasoning. SEER effectively addresses the key limitations of previous work: limited exploration of reasoning paths, lack of quality assessment for the intermediate reasoning step, and the negative impact of “overthinking”. Experimental results demonstrate that SEER effectively improves code generation

performance across multiple benchmarks, enabling smaller models to outperform larger ones.

Acknowledgments

The work described in this paper was supported by Research Grants Council of the Hong Kong Special Administrative Region, China (No. CUHK 14209124 of the General Research Fund) and Guangdong-Hong Kong Technology Cooperation Funding (Ref. No. GHP/146/22SZ). The work was also supported by National Natural Science Foundation of China under project (No. 62472126), Shenzhen-Hong Kong Jointly Funded Project (Category A, No. SGDX20230116 091246007), CCF-Huawei Populus Grove Fund.

References

- [1] Rohan Anil, Sebastian Borgeaud, Yonghui Wu, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M. Dai, Anja Hauth, Katie Millican, David Silver, Slav Petrov, Melvin Johnson, Ioannis Antonoglou, Julian Schrittwieser, Amelia Glaese, Jilin Chen, Emily Pitler, Timothy P. Lillicrap, Angeliki Lazaridou, Orhan Firat, James Molloy, Michael Isard, Paul Ronald Barham, Tom Hennigan, Benjamin Lee, Fabio Viola, Malcolm Reynolds, Yuanzhong Xu, Ryan Doherty, Eli Collins, Clemens Meyer, Eliza Rutherford, Erica Moreira, Kareem Ayoub, Megha Goel, George Tucker, Enrique Piqueras, Maxim Krikun, Iain Barr, Nikolay Savinov, Ivo Danihelka, Becca Roelofs, Anaïs White, Anders Andreassen, Tamara von Glehn, Lakshman Yagati, Mehran Kazemi, Lucas Gonzalez, Misha Khalman, Jakub Sygnowski, and et al. 2023. Gemini: A Family of Highly Capable Multimodal Models. *CoRR abs/2312.11805* (2023).
- [2] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. 2021. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732* (2021).
- [3] Hao Bai, Yifei Zhou, Mert Cemri, Jiayi Pan, Alane Suhr, Sergey Levine, and Aviral Kumar. 2024. Digirl: Training in-the-wild device-control agents with autonomous reinforcement learning. *arXiv preprint arXiv:2406.11896* (2024).
- [4] Graeme Best, Oliver M Cliff, Timothy Patten, Ramgopal R Mettu, and Robert Fitch. 2019. Dec-MCTS: Decentralized planning for multi-robot active perception. *The International Journal of Robotics Research* 38, 2-3 (2019), 316–337.
- [5] Edward Y. Chang, Yuxuan Tong, Morry Niu, Graham Neubig, and Xiang Yue. 2025. Demystifying Long Chain-of-Thought Reasoning in LLMs. *CoRR abs/2502.03373* (2025).
- [6] ChatGPT. 2022. ChatGPT. <https://chat.openai.com/>.
- [7] Guoxin Chen, Mimpeng Liao, Chengxi Li, and Kai Fan. 2024. AlphaMath Almost Zero: Process Supervision without Process. (2024).
- [8] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).
- [9] Runjin Chen, Zhenyu Zhang, Junyuan Hong, Souvik Kundu, and Zhangyang Wang. 2025. SEAL: Steerable Reasoning Calibration of Large Language Models for Free. *CoRR abs/2504.07986* (2025).
- [10] Wenhui Chen, Xueguang Ma, Xinyi Wang, and William W. Cohen. 2023. Program of Thoughts Prompting: Disentangling Computation from Reasoning for Numerical Reasoning Tasks. *Trans. Mach. Learn. Res.* 2023 (2023).
- [11] Rémi Coulom. 2006. Efficient selectivity and backup operators in Monte-Carlo tree search. In *International conference on computers and games*. Springer, 72–83.
- [12] Yihong Dong, Xue Jiang, Zhi Jin, and Ge Li. 2024. Self-Collaboration Code Generation via ChatGPT. *ACM Trans. Softw. Eng. Methodol.* 33, 7 (2024), 189:1–189:38.
- [13] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, Anirudh Goyal, Anthony Hartshorn, Aobo Yang, Archi Mitra, Archie Sravankumar, Artem Korenev, Arthur Hinsvark, Arun Rao, Aston Zhang, Aurélien Rodriguez, Austen Gregerson, Ava Spataru, Baptiste Rozière, Bethany Biron, Binh Tang, Bobbie Chern, Charlotte Caucheteux, Chaya Nayak, Chloe Bi, Chris Marra, Chris McConnell, Christian Keller, Christophe Touret, Chunyang Wu, Corinne Wong, Cristian Canton Ferrer, Cyrus Nikolaidis, Damien Allonsius, Daniel Song, Danielle Pintz, Danny Livshits, David Esiobu, Dhruv Choudhary, Dhruv Mahajan, Diego Garcia-Olano, Diego Perino, Dieuwke Hupkes, Egor Lakomkin, Ehab AlBadawy, Elina Lobanova, Emily Dinan, Eric Michael Smith, Filip Radenovic, Frank Zhang, Gabriel Synnaeve, Gabrielle Lee, Georgia Lewis Anderson, Graeme Nail, Grégoire Mialon, Guan Pang, Guillem Cucurell, Hailey Nguyen, Hannah Korevaar, Hu Xu, Hugo Touvron, Iliyan Zarov, Imanol Arrieta Ibarra, Isabel M. Kloumann, Ishan Misra, Ivan Evtimov, Jade Copet, Jaewon Lee, Jan Geffert, Jana
- [14] Eugene A Feinberg and Adam Shwartz. 2012. *Handbook of Markov decision processes: methods and applications*. Vol. 40. Springer Science & Business Media.
- [15] Shuzheng Gao, Cuiyun Gao, Wenchao Gu, and Michael R. Lyu. 2025. Search-Based LLMs for Code Optimization. In *47th IEEE/ACM International Conference on Software Engineering, ICSE 2025, Ottawa, ON, Canada, April 26 - May 6, 2025*. IEEE, 578–590.
- [16] Shuzheng Gao, Cuiyun Gao, Yulan He, Jichuan Zeng, Lunyiu Nie, Xin Xia, and Michael R. Lyu. 2023. Code Structure-Guided Transformer for Source Code Summarization. *ACM Trans. Softw. Eng. Methodol.* 32, 1 (2023), 23:1–23:32.
- [17] Shuzheng Gao, Wenxin Mao, Cuiyun Gao, Li Li, Xing Hu, Xin Xia, and Michael R. Lyu. 2024. Learning in the Wild: Towards Leveraging Unlabeled Data for Effectively Tuning Pre-trained Code Models. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering, ICSE 2024, Lisbon, Portugal, April 14-20, 2024*. ACM, 80:1–80:13.
- [18] Shuzheng Gao, Chaozheng Wang, Cuiyun Gao, Xiaoqian Jiao, Chun Yong Chong, Shan Gao, and Michael R. Lyu. 2025. The Prompt Alchemist: Automated LLM-Tailored Prompt Optimization for Test Case Generation. *CoRR abs/2501.01329* (2025).
- [19] Shuzheng Gao, Xin-Cheng Wen, Cuiyun Gao, Wenxuan Wang, Hongyu Zhang, and Michael R. Lyu. 2023. What Makes Good In-Context Demonstrations for Code Intelligence Tasks with LLMs? In *38th IEEE/ACM International Conference on Automated Software Engineering, ASE 2023, Luxembourg, September 11-15, 2023*. IEEE, 761–773.
- [20] Shuzheng Gao, Hongyu Zhang, Cuiyun Gao, and Chaozheng Wang. 2023. Keeping Pace with Ever-Increasing Data: Towards Continual Learning of Code Intelligence Models. In *45th IEEE/ACM International Conference on Software Engineering, ICSE 2023, Melbourne, Australia, May 14-20, 2023*. IEEE, 30–42.
- [21] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shiron Ma, Peiyi Wang, Xiao Bi, et al. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948* (2025).
- [22] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Y Wu, YK Li, et al. 2024. DeepSeek-Coder: When the Large Language Model Meets Programming—The Rise of Code Intelligence. *arXiv preprint arXiv:2401.14196* (2024).
- [23] Zeyu Han, Chao Gao, Jinyang Liu, Jeff Zhang, and Sai Qian Zhang. 2024. Parameter-efficient fine-tuning for large models: A comprehensive survey. *arXiv preprint arXiv:2403.14608* (2024).
- [24] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022*. OpenReview.net.
- [25] Binyuan Hui, Jian Yang, Zeyu Cui, Jiayi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, et al. 2024. Qwen2. 5-coder technical report. *arXiv preprint arXiv:2409.12186* (2024).
- [26] icse26 730. 2025. Replicate Package. <https://github.com/shuzhenggao/ICSE26SEER>.
- [27] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. 2024. Livecodebench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974* (2024).
- [28] Naman Jain, Skanda Vaidyanath, Arun Iyer, Nagarajan Natarajan, Suresh Parthasarathy, Sriram K. Rajamani, and Rahul Sharma. 2022. Jigsaw: Large Language Models meet Program Synthesis. In *44th IEEE/ACM 44th International Conference on Software Engineering, ICSE 2022, Pittsburgh, PA, USA, May 25-27, 2022*. ACM, 1219–1231.
- [29] Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de Las Casas, Emma Bou Hanna, Florian Bressand, Gianna Lengyel, Guillaume Bour, Guillaume Lample, Léo Renard Lavaud, Lucile Saulnier, Marie-Anne Lachaux, Pierre Stock, Sandeep Subramanian, Sophia Yang, Szymon Antoniak, Teven Le Scao, Théophile Gervet, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. 2024. Mixture of Experts. *CoRR abs/2401.04088* (2024).
- [30] Xue Jiang, Yihong Dong, Lecheng Wang, Zheng Fang, Qiwei Shang, Ge Li, Zhi Jin, and Wenpin Jiao. 2024. Self-Planning Code Generation with Large Language Models. *ACM Trans. Softw. Eng. Methodol.* 33, 7 (2024), 182:1–182:30.
- [31] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. 2022. Large Language Models are Zero-Shot Reasoners. In *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*.

- [32] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. 2015. Deep learning. *nature* 521, 7553 (2015), 436–444.
- [33] Jia Li, Ge Li, Yongmin Li, and Zhi Jin. 2023. Structured chain-of-thought prompting for code generation. *ACM Transactions on Software Engineering and Methodology* (2023).
- [34] Jia Li, Yongmin Li, Ge Li, Zhi Jin, Yiyang Hao, and Xing Hu. 2023. SkCoder: A Sketch-based Approach for Automatic Code Generation. In *45th IEEE/ACM International Conference on Software Engineering, ICSE 2023, Melbourne, Australia, May 14–20, 2023*. IEEE, 2124–2135.
- [35] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. 2023. Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10–16, 2023*, Alice Oh, Tristan Naumann, Amir Globerson, Kate Saenko, Moritz Hardt, and Sergey Levine (Eds.).
- [36] Anton Lozhkov, Raymond Li, Loubna Ben Allal, Federico Cassano, Joel Lamy-Poirier, Nouamane Tazi, Ao Tang, Dmytro Pykhtar, Jiawei Liu, Yuxiang Wei, Tianyang Liu, Max Tian, Denis Kocetkov, Arthur Zucker, Younes Belkada, Zijian Wang, Qian Liu, Dmitry Abulkhanov, Indraneil Paul, Zhuang Li, Wen-Ding Li, Megan Risdal, Jia Li, Jian Zhu, Terry Yue Zhuo, Evgenii Zheltonozhskii, Nii Osae Osae Dade, Wenhao Yu, Lucas Krauß, Naman Jain, Yixuan Su, Xuanli He, Manan Dey, Edoardo Abati, Yekun Chai, Niklas Muennighoff, Xiangru Tang, Muh-tasham Oblokulov, Christopher Akiki, Marc Marone, Chenghao Mou, Mayank Mishra, Alex Gu, Binyuan Hui, Tri Dao, Armel Zebaze, Olivier Dehaene, Nicolas Patry, Canwen Xu, Julian J. McAuley, Han Hu, Torsten Scholak, Sébastien Paquet, Jennifer Robinson, Carolyn Jane Anderson, Nicolas Chapados, et al. 2024. StarCoder 2 and The Stack v2: The Next Generation. *CoRR abs/2402.19173* (2024).
- [37] Ziyang Luo, Can Xu, Pu Zhao, Qingfeng Sun, Xiubo Geng, Wenxiang Hu, Chongyang Tao, Jing Ma, Qingwei Lin, and Daxin Jiang. 2024. WizardCoder: Empowering Code Large Language Models with Evol-Instruct. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7–11, 2024*. OpenReview.net.
- [38] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10–16, 2023*.
- [39] Michael McCloskey and Neal J Cohen. 1989. Catastrophic interference in connectionist networks: The sequential learning problem. In *Psychology of learning and motivation*. Vol. 24. Elsevier, 109–165.
- [40] open r1. 2025. Open R1. <https://github.com/huggingface/open-r1/tree/main>.
- [41] OpenAI. 2023. GPT-4 Technical Report. *CoRR abs/2303.08774* (2023).
- [42] openai. 2025. OpenAI. <https://openai.com/index/introducing-o3-and-o4-mini/>.
- [43] Christopher D Rosin. 2011. Multi-armed bandits with episode context. *Annals of Mathematics and Artificial Intelligence* 61, 3 (2011), 203–230.
- [44] Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xi-aoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, Artyom Kozhevnikov, Ivan Evtimov, Joanna Bitton, Manish Bhatt, Cristian Canton-Ferrer, Aaron Grattafiori, Wenhan Xiong, Alexandre Défossez, Jade Copet, Faisal Azhar, Hugo Touvron, Louis Martin, Nicolas Usunier, Thomas Scialom, and Gabriel Synnaeve. 2023. Code Llama: Open Foundation Models for Code. *CoRR abs/2308.12950* (2023).
- [45] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10–16, 2023*.
- [46] David Silver, Aja Huang, Chris J Maddison, Arthur Guez, Laurent Sifre, George Van Den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, et al. 2016. Mastering the game of Go with deep neural networks and tree search. *nature* 529, 7587 (2016), 484–489.
- [47] David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, et al. 2017. Mastering the game of go without human knowledge. *nature* 550, 7676 (2017), 354–359.
- [48] Chaozheng Wang, Jia Feng, Shuzheng Gao, Cuiyun Gao, Zongjie Li, Ting Peng, Hailiang Huang, Yuetang Deng, and Michael R Lyu. 2025. Beyond PEFT: Layer-Wise Optimization for More Effective and Efficient Large Code Model Tuning. *Proc. ACM Softw. Eng.* 2, FSE (2025), 1567–1590.
- [49] Chaozheng Wang, Shuzheng Gao, Cuiyun Gao, Wenxuan Wang, Chun Yong Chong, Shan Gao, and Michael R Lyu. 2024. A systematic evaluation of large code models in api suggestion: When, which, and how. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 281–293.
- [50] Chaozheng Wang, Zongjie Li, Cuiyun Gao, Wenxuan Wang, Ting Peng, Hailiang Huang, Yuetang Deng, Shuai Wang, and Michael R Lyu. 2024. Exploring multi-lingual bias of large code models in code generation. *arXiv preprint arXiv:2404.19368* (2024).
- [51] Chaozheng Wang, Zezhou Yang, Shuzheng Gao, Cuiyun Gao, Ting Peng, Hailiang Huang, Yuetang Deng, and Michael Lyu. 2025. Rag or fine-tuning? a comparative study on lcms-based code completion in industry. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*. 93–104.
- [52] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28–December 9, 2022*.
- [53] Yuxiang Wei, Federico Cassano, Jiawei Liu, Yifeng Ding, Naman Jain, Zachary Mueller, Harm de Vries, Leandro von Werra, Arjun Guha, and Lingming Zhang. 2024. SelfCodeAlign: Self-Alignment for Code Generation. In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10–15, 2024*.
- [54] Yuxiang Wei, Zhe Wang, Jiawei Liu, Yifeng Ding, and Lingming Zhang. 2024. Magicoder: Empowering Code Generation with OSS-Instruct. In *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21–27, 2024*. OpenReview.net.
- [55] Xin-Cheng Wen, Xincheng Wang, Yujia Chen, Ruida Hu, David Lo, and Cuiyun Gao. 2024. VulEval: Towards Repository-Level Evaluation of Software Vulnerability Detection. *CoRR abs/2404.15596* (2024). <https://doi.org/10.48550/arXiv.2404.15596>
- [56] Xin-Cheng Wen, Yijun Yang, Cuiyun Gao, Yang Xiao, and Deheng Ye. 2025. Boosting Vulnerability Detection of LLMs via Curriculum Preference Optimization with Synthetic Reasoning Data. In *Findings of the Association for Computational Linguistics, ACL 2025, Vienna, Austria, July 27–August 1, 2025*, Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (Eds.). Association for Computational Linguistics, 8935–8949.
- [57] Xin-Cheng Wen, Zirui Lin, Yijun Yang, Cuiyun Gao, and Deheng Ye. 2025. Vul-R2: A Reasoning LLM for Automated Vulnerability Repair. *arXiv preprint arXiv:2510.05480* (2025).
- [58] Shijie Xia, Yiwei Qin, Xuefeng Li, Yan Ma, Run-Ze Fan, Steffi Chern, Haoyang Zou, Fan Zhou, Xiangkun Hu, Jiahe Jin, Yanheng He, Yixin Ye, Yixiu Liu, and Pengfei Liu. 2025. Generative AI II: Test Time Scaling Drives Cognition Engineering. *CoRR abs/2504.13828* (2025).
- [59] Yuxi Xie, Anirudh Goyal, Wenyue Zheng, Min-Yen Kan, Timothy P. Lillicrap, Kenji Kawaguchi, and Michael Shieh. 2024. Monte Carlo Tree Search Boosts Reasoning via Iterative Preference Learning. *CoRR abs/2405.00451* (2024).
- [60] Guang Yang, Yu Zhou, Xiang Chen, Xiangyu Zhang, Terry Yue Zhuo, and Taolue Chen. 2024. Chain-of-Thought in Neural Code Generation: From and for Lightweight Language Models. *IEEE Trans. Software Eng.* 50, 9 (2024), 2437–2457.
- [61] Wang Yang, Xiang Yue, Vipin Chaudhary, and Xiaotian Han. 2025. Speculative Thinking: Enhancing Small-Model Reasoning with Large Model Guidance at Inference Time. *CoRR abs/2504.12329* (2025).
- [62] Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D. Goodman. 2022. STaR: Bootstrapping Reasoning With Reasoning. In *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28–December 9, 2022*.
- [63] Dan Zhang, Sining Zhoubian, Yisong Yue, Yuxiao Dong, and Jie Tang. 2024. ReST-MCTS*: LLM Self-Training via Process Reward Guided Tree Search. *CoRR abs/2406.03816* (2024).
- [64] Huan Zhang, Wei Cheng, Yuhan Wu, and Wei Hu. 2024. A Pair Programming Framework for Code Generation via Multi-Plan Exploration and Feedback-Driven Refinement. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, ASE 2024, Sacramento, CA, USA, October 27–November 1, 2024*. ACM, 1319–1331.
- [65] Kechi Zhang, Zhuo Li, Jia Li, Ge Li, and Zhi Jin. 2023. Self-Edit: Fault-Aware Code Editor for Code Generation. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2023, Toronto, Canada, July 9–14, 2023*. Association for Computational Linguistics, 769–787.
- [66] Qiyan Zhang, Fuyuan Lyu, Zexu Sun, Lei Wang, Weixu Zhang, Zhihan Guo, Yufei Wang, Irwin King, Xue Liu, and Chen Ma. 2025. What, How, Where, and How Well? A Survey on Test-Time Scaling in Large Language Models. *CoRR abs/2503.24235* (2025).
- [67] Ziyao Zhang, Yanli Wang, Chong Wang, Jiachi Chen, and Zibin Zheng. 2024. LLM Hallucinations in Practical Code Generation: Phenomena, Mechanism, and Mitigation. *CoRR abs/2409.20550* (2024).
- [68] Denny Zhou, Nathanael Schärli, Le Hou, Jason Wei, Nathan Scales, Xuezhi Wang, Dale Schuurmans, Claire Cui, Olivier Bousquet, Quoc V. Le, and Ed H. Chi. 2023. Least-to-Most Prompting Enables Complex Reasoning in Large Language Models. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1–5, 2023*. OpenReview.net.
- [69] Qihao Zhu, Daya Guo, Zhihong Shao, Dejian Yang, Peiyi Wang, Runxin Xu, Y Wu, Yukun Li, Huazuo Gao, Shirong Ma, et al. 2024. Deepseek-coder-v2: Breaking the barrier of closed-source models in code intelligence. *arXiv preprint arXiv:2406.11931* (2024).